

KMI/KOM - Komprese dat

Poznámky z výuky (2025 – 2026)
Verze z 09. 06. 2026

Vojtěch Netrh
vojtanetrh@gmail.com

Obsah

1 Režijní informace	3
1.1 Státnicové okruhy	3
1.2 Zkouškové otázky	3
2 Komprese dat, její techniky a metody	4
2.1 Fyzický model dat	5
2.2 Pravděpodobnostní model dat	5
2.3 Markovův model dat	5
2.4 Další modely dat	5
3 Teorie informace	5
3.1 Klasická (Shannonova) teorie	6
3.1.1 Entropie	6
3.2 Algoritmická (Kolmogorova) teorie	6
4 Kódování	6
4.1 Optimální kód	7
5 Základní techniky a kódování čísel	8
5.1 Run-length kódování (RLE)	8
5.1.1 Diferenční kódování	9
5.2 Move-to-front kódování	9
5.3 Kódování čísel	9
5.3.1 Unární kódování	9
5.3.2 Eliasovy kódy	9
5.3.3 Fibonacciho kódy	10
5.3.4 Golombovy kódy	10
5.3.5 Riceovy kódy	10
6 Statistické metody	11
6.1 Tunstalovo kódování	11
6.2 Shannon-Fanovo kódování	11
6.3 Huffmanovo kódování	12
6.3.1 Adaptivní Huffmanovo kódování	14
6.4 Aritmetické kódování	15
6.5 QM kódování	16
7 Kontextové metody	17
7.1 PPM (Prediction with Partial Matching)	17
7.2 PAQ	18
7.3 Blokové třídění (Burrows-Wheelerova transformace)	18
8 Slovníkové metody	20
8.1 Rodina LZ77	20
8.2 Rodina LZ78	21
8.2.1 Varianta LZW	21

1 Režijní informace

Vyučující: Jan Outrata

Otázky jsou dány tím co probereme v přednáškách, asi to co má na stránkách rozvržené do 7 kapitol.

1.1 Státnicové okruhy

1. Základní pojmy, taxonomie metod, míry komprese, typy modelů dat, pravděpodobnostní a Markovův model dat.
2. Run-length encoding a Move-to-front kódování.
3. Kódování čísel: unární kód, Eliasovy, Fibonacciho a Golombovy kódy.
4. Tunstallův kód a Shannon-Fanovo kódování.
5. Huffmanovo kódování se semi-adaptivním modelem.
6. Huffmanovo kódování s adaptivním modelem.
7. Aritmetické kódování.
8. Kontextové kódování (PPM).
9. Blokové třídění.
10. Třída slovníkových metod LZ77.
11. Třída slovníkových metod LZ78, reprezentace slovníku.
12. Slovníková metoda LZW.

1.2 Zkouškové otázky

1. Taxonomie kompresních metod, modely dat (pravděpodobnostní, Markovův).
2. Potřebné pojmy z teorie informace a kódování (entropie, optimální prefixový kód)
3. Základní techniky (RLE, MTF) a kódování čísel (Eliasovy kódy).
4. Statistické metody: Shannon-Fanovo a Huffmanovo kódování, principy a implementace.
5. Statistické metody: Aritmetické a QM kódování, principy a implementace.
6. Kontextové metody: Metody PPM a PAQ (context mixing), principy a implementace.
7. Kontextové metody: Blokové třídění (Burrows-Wheelerova transformace, BWT), principy a implementace.
8. Slovníkové metody: Rodina metod LZ77 a varianta Deflate, principy a implementace.
9. Slovníkové metody: Rodina metod LZ78 a varianta LZW, principy a implementace.
10. Další bezztrátové metody: Gramatické, statistické a jiné vybrané metody.

2 Komprese dat, její techniky a metody

Jedná se o proces, při kterém se zmenší velikost informačního obsahu, ale zachová se daná míra obsažené informace. Jedná se dost o experimentální obor. Šetří místo (vhodné pro ukládání i přenos). Dnes je samozřejmostí úplně všude.

Příklady z minulosti: Brailovo písmo, morseovka.

Proces se dá rozdělit na 2 fáze:

1. Identifikování a modelování struktury dat (s vynecháním redundancí)
2. Kódování dat podle modelu

Strukturou se bere opakování vzorů (takže statistická = frekvence atd.), korelace mezi vzory. Pro různé části dat můžeme mít různé modely (nemusíme to dělat celé najednou).

Příklad 1

$x_1, x_2, \dots, x_{12} = 2, 2, 4, 6, 7, 7, 7, 10, 10, 11, 11, 14$

1. čísla od 12 do 14 binárně $\Rightarrow$ 48b
2. 7 různých čísel binárně $\Rightarrow$ 36b
3. častější číslo = kratší kód $\Rightarrow$ **0I** pro 7, **III** pro 11, **II0** pro 10, ... $\Rightarrow$ 33b
4. kódování opakování čísla 32b
5. malé rozdíly mezi sousedními čísly $\leadsto$ predikce $\Rightarrow$ 26b
6. vztah mezi čísly $\leadsto$ predikce $\Rightarrow$ 19b

Zachováním veškeré zachované informace se myslí: odstranění redundance (obsah co nemá žádnou „hodnotu“).

Pro kompresi datu potřebujeme vždy 2 části skládanky, kompresní algoritmus, abychom byli schopni data zmenšit a dekompresní algoritmus (rekonstrukční), abychom je byli schopni obnovit (obvykle po přenosu).

Bezztrátová komprese

Dekomprimovaná data jsou stejná jako originální (nic se neztratí). Vhodné pro text, programové soubory, citlivé záznamy. V tomto kurzu a materiálu se zabýváme výhradně bezztrátovou kompresí. Např. Huffmanovo kódování, aritmetické kódování, PPM, BWT, ...

Ztrátová komprese

Při kompresi se nějaká data vynechají (dekomprimovaná data nejsou totožná s originálními). Díky této ztrátě je poskytnuta vyšší komprese. Hodí se pro obraz, video, zvuk, ... Vzorkování a kvantizace, diferenční kódování, transformační a podpásmové kódování. Používá se v řadě klasických typech souborů (JPEG).

Míry kompresních algoritmů

Poznámka 2

Kompresní algoritmy jsou lineární a konstantní, případně logaritmické.

Zkoumá se:

- **compression ratio** – poměr velikosti originálních a komprimovaných dat
- **compression rate** – průměrná velikost komprimovaných dat na vzorek originálních
- **distortion** – míra zkreslení dat

2.1 Fyzický model dat

Popis zdroje nebo procesu generování dat z hlediska fyzického fungování. Obecně je příliš složitý až nemožný.

2.2 Pravděpodobnostní model dat

Pravděpodobnost výskytu symbolů dat je nezávislá na výskytu ostatních symbolů (jedná se o nejjednodušší předpoklad). Statistický popis se zjistí empiricky (pozorováním). Používá se pro statistické bezztrátové kompresní metody. Používá se tzv. *klasická pravděpodobnost*.

2.3 Markovův model dat

Výskyt symbolu x_j závisí na výskytu předchozích symbolů x_i (kde $i < j$). Vychází z pravděpodobnostního modelu. Tohoto modelu mohou být různé řady (od nultého). Vychází z *Markovova řetězce*, který pro k -tý Markovův model je:

$$P(x_n | x_{n-1}, \dots, x_{n-k}) = P(x_n | x_{n-1}, \dots, x_{n-k}, \dots)$$

Pokud má abeceda počet symbolů l , pak l^k je počet stavů. Pro *první Markovův model* platí

$$P(x_n | x_{n-1}) = P(x_n | x_{n-1}, x_{n-2}, x_{n-3}, \dots)$$

Obecně modely vyšších k řádů mají vyšší míru komprese než nezávislé výskyty symbolů.

Markovův model

Příklad 3

$$x_1 x_2 \dots x_{10} = aababbabaa$$

stavy modelu 1. řádu = posloupnosti (bezprostředně) předchozích symbolů délky 1 pro všechny symboly: a, b

$$P(a) = \frac{5}{9}, P(b) = \frac{4}{9}, \\ P(a|a) = \frac{2}{5}, P(b|a) = \frac{3}{5}, P(a|b) = \frac{3}{4}, P(b|b) = \frac{1}{4}$$

stavy modelu 2. řádu = posloupnosti (bezprostředně) předchozích symbolů délky 2 pro všechny symboly: aa, ab, ba, bb

$$P(aa) = \frac{1}{8}, P(ab) = \frac{3}{8}, P(ba) = \frac{3}{8}, P(bb) = \frac{1}{8}, \\ P(a|aa) \rightarrow 0, P(b|aa) \rightarrow 1, P(a|ab) = \frac{2}{3}, P(b|ab) = \frac{1}{3}, P(a|ba) = \frac{1}{3}, P(b|ba) = \frac{2}{3}, \\ P(a|bb) \rightarrow 1, P(b|bb) \rightarrow 0$$

Je tam jen 9 ve jmenovateli u jednoho symbolu a to z důvodu, že prvnímu symbolu nic nepředchází (celkově má řetěze tedy délku 10).

2.4 Další modely dat

Slovníkový – odkazy do paměti předchozích symbolů v datech

Prediktivní – predikce symbolů na základě okolních (nikoliv jen předchozích) symbolů v datech

3 Teorie informace

Jedná se o vědu, jak měřit, ukládat a přenášet data. Řeší problém náhodnosti a nejistoty pokud se bavíme o tom „jak moc informací“ je uloženo v datech. Informace totiž nemůže být měřena

pouze její velikostí (třeba kolik místa zabírá na disku). Její zakladatelem je Claude Shannon. Používá se jako rámec pro bezztrátové kompresní metody. Úzce souvisí s pravděpodobnostním modelem dat.

3.1 Klasická (Shannonova) teorie

Míra průměrná informace experimentu = výsledky experimentu (data nebo jevy). Informaci jevu A značíme jako $i(A)$, její pravděpodobnost je $P(A)$. Potom definujeme míru její informace jako

$$i(A) = -\log_b P(A)$$

Jednotka b je pro: 2 – bit (tu používá Shannon), e – nat, 10 – hartley.

3.1.1 Entropie

Jde o průměrnou hodnotu vlastní informace pro všechny možné výstupy náhodného zdroje dat. Značí se H .

$$H(A_i) = \sum_i P(A_i) i(A_i) = - \sum_i P(A_i) \log P(A_i)$$

Shannon dokázal že pouze toto splňuje 3 požadavky:

1. spojitá funkce
2. pro stejně pravděpodobné jevy musí monotónně růst s počtem možností
3. musí být stejná při rozdělení na k podexperimentů

Shannon objevil tzv. *Noiseless Source Coding Theorem*, který říká, že entropie představuje (teoretickou) spodní hranici pro bezztrátovou kompresi. Čili pokud si entropii spočítáme, víme že se nikdy nemůžeme dostat pro výsledný počet bitů na číslo. Dá se aplikovat i na přirozený jazyk, kde to jaké písmeno bude následující je silně dáno předchozím znakem (např. často po q následuje u).

[Příklady v prezentacích]

3.2 Algoritmická (Kolmogorova) teorie

Základem jak plyne z názvu je *Kolmogorova složitost* $K(x)$. Jde o velikost nejkratšího algoritmu (programu) potřebného k vygenerování posloupnosti x . Velikost tohoto programu by teoreticky měla odpovídat nejvyšší míře komprese, které jde dosáhnout. Avšak neexistuje žádný systematický způsob jak tento algoritmus sestavit nebo se k němu alespoň přiblížit.

4 Kódování

Základem kódování je kód, takže musíme znát abecedu a její symboly. Známe např. blokový kód, kde všechna slova mají stejnou délku (v praxi ASCII).

Jednoznačně dekódovatelný kód

Definice 4

Každá neprázdná posloupnost symbolů z kódované abecedy je zřetězením nejvýše jedné posloupnosti kódových slov.

Prostě že daný kód můžeme namapovat jen na jediný řetězec.

Každý blokový je jednoznačný. Konkrétní příklady: $\{0, 01, 011, 111\}$ **ano**, $\{0, 01, 10, 11\}$ **ne**.

Prefixový kód

Definice 5

Žádné kódové slovo není prefixem jiného kódového slova. Hezky se sestavuje za pomoci stromu.

Např. $\{0, 10, 110, 111\}$

Z každého jednoznačně dekódovatelného kódu, který není prefixový můžeme prefixový vytvořit a bude mít stejnou délku kódových slov.

Kraftova věta

Theorem 6

Prefixový kód s k kódovými slovy délek $l_1, l_2, \dots, l_k$ nad kódovou abecedou velikosti m existuje právě když

$$\sum_{i=1}^k m^{-l_i} \leq 1$$

Vzorec se nazývá *Kraftovou nerovností*.

Umožňuje nám zjistit zda-li daný kód může existovat. Stačí nám znát velikost abecedy a to jak dlouhá slova použijeme. Vlastně se jedná o to, že nějaký celek (to je ta 1) můžeme rozdělit jen na určitý počet částí, které nepřevyšší jediný celek.

4.1 Optimální kód

Pro pravděpodobnostní model dat jde o prefixový kód. Můžeme vypočítat *průměrnou délku kódu* (code rate) pomocí

$$\bar{l}(C(A)) = \sum_{i=1}^n P(a_i) l(a_i)$$

pro všechny $a \in A$, kde $P(a_i) \neq 0$ je pravděpodobnost výskytu symbolu a_i , $l(a_i)$ je délka kódového slova.

Pro průměrnou délku kódu máme horní (1) i dolní (2) limity. m je velikost kódové abecedy.

$$(1) \frac{H(A)}{\log_b(m)} + 1$$

$$(2) \frac{H(A)}{\log_b(m)}$$

Poznámka 7

Pokud $\bar{l}(C(A)) = \frac{H(A)}{\log_b(m)}$, pak se jedná o optimální kód. Dále definováno v Shannon noiseless coding teorému.

Shannon noiseless coding theorem

Theorem 8

Pro optimální prefixový kód $C(A)$ ze zdrojové abecedy A do kódové abecedy B platí

$$\frac{H(A)}{\log_b(m)} \leq \bar{l}(C(A)) < \frac{H(A)}{\log_b(m)} + 1$$

kde je:

- $H(A)$... entropie zdroje symbolů z A
- m ... velikost B
- b je stejné jako v H

Věta

Theorem 9

Pro optimální prefixový kód ze zdrojové abecedy A do kódové abecedy B platí:

1. Symboly z A s větší pravděpodobností výskytu mají kratší kódová slova
2. $m' \in \{2, 3, \dots, m\}$; $m' \equiv n \pmod{m-1}$ symbolů z A s nejmenší pravděpodobností výskytu, kde $n \leq 2$ je velikost A a $m \leq 2$ je velikost B , má stejně dlouhá (maximální) kódová slova, která se liší v jediném symbolu

McMillanova věta

Theorem 10

Jednoznačně dekódovatelný kód s k kódovými slovy délek $l_1, l_2, \dots, l_k$ nad kódovanou abecedou velikosti m existuje právě když

$$\sum_{i=1}^k m^{-l_i} \leq 1$$

Jde pouze o mírnou úpravu Kraftovy věty. Vlastně z ní vychází, jelikož prefixový kód je jednoznačně dekódovatelný.

5 Základní techniky a kódování čísel

5.1 Run-length kódování (RLE)

Kóduje posloupnosti stejných zdrojových symbolů (označená jako **runs**) do jediného symbolu a čísla udávajícího počet opakování. Je potřeba i koeficient k , který říká od kolika symbolů kompresi provedeme. Někdy se jednotlivé runs oddělují speciálním kódem příznaku. Tento příznak by neměl být zaměnitelný se symbolem v kódování (používá se třeba dolar \$).

Používá se v textu a obrazu (BMP).

RLE

Příklad 11

- originální posloupnost: **WWWWWWAAARRRR**
- po RLE kompresi: **6WAA4R**
- k není přesně stanoveno, ale může být max 3

Pseudokód algoritmu se dá určitě vymyslet z hlavy, není to nic složitého. Stejně jako dekódování přijaté sekvence.

5.1.1 Diferenční kódování

Místo absolutního kódování hodnoty zapisujeme pouze rozdíl, o který se hodnota změnila od předchozí. Používá se třeba u snímků ve videu nebo časových řad.

5.2 Move-to-front kódování

Jedná se spíše o datovou transformaci. Vstupní posloupnost dat se přeskládá, aby v ní bylo více opakujících se vzorů.

MTF kódování

Příklad 12

Máme vstup $L = \text{banana}$ a abecedu $A = [a, b, n, r]$

1. Očíslujeme znaky v abecedě $\{a : 0\}, \{b : 1\}, \dots$ (jsou to indexy)
2. Vezmeme první znak na vstupu (b), najdeme jeho číslo, dáme ho na výstup a daný znak přesuneme na začátek očíslovaného pole
3. Takto opakujeme pro celou posloupnost

Takže **vstup** – **výstup** – **posloupnost** a její změna:

- $b - 1 - [a, b, n, r] \rightarrow [b, a, n, r]$
- $a - 1 - [b, a, n, r] \rightarrow [a, b, n, r]$
- $n - 2 - [a, b, n, r] \rightarrow [n, a, b, r]$
- $\vdots$

Při dekódování bychom brali seznam abecedy ve stejném pořadí a podle číslic vybírali písmena a rekonstruovali řetězec. Nic se nemusí obracet.

5.3 Kódování čísel

Jak zobrazit všechna celá čísla bijektivně na přirozená?

Předpokládáme, že větší čísla mají nižší pravděpodobnost výskytu. Používají se kódy s proměnnou délkou (tzv. *variable-length codes* a *fixed-to-variable code*).

5.3.1 Unární kódování

Kóduje přirozená čísla. Pokud máme číslo 5, tak $5 \times$ zřetězíme **1** a zakončíme nulou. Takže pro 5 to bude **111110**.

5.3.2 Eliasovy kódy

Několik druhů pojmenovaných jako alpha, beta, gamma, delta, omega (ta je rekurzivní). Kóduje přirozená čísla.

1. Alfa α kód – prostě unární kód
2. Beta β kód – binární reprezentace čísla
3. Gamma γ kód – kombinace délky v binární soustavě a binárního zápisu
4. Delta δ kód – podobný jako gamma, ale délka se zapíše pomocí gamma kódu a připojí se binární zápis čísla bez první **1**
5. Omega ω kód – rekurzivní přístup, postu je lehce složitější

Postup pro gamma γ metodu:

1. zjistíme počet bitů nutný pro zápis čísla x v binární soustavě (značeno n)
2. jako prefix zapíšeme $n - 1$ nul a za ně jedinou jedničku
3. jako suffix napíšeme ono číslo v binárce, ale vynecháme první jedničku

- např.: $5 = 101_2$, takže 3 bity; prefix budou 2 nuly a 1 jednička (**001**); suffix bude **01**; výsledek po spojení je **00101**

Postup pro omega ω metodu:

1. na konec výstupu zapíšeme **0**
2. ze vstupního čísla x uděláme binární reprezentaci a tu dáme před **0** na konci
3. spočítáme délku binárního zápisu N , odečteme 1 a z toho čísla uděláme x (pak $x = N - 1$)
4. pokračujeme od 2. kroku

5.3.3 Fibonacciho kódy

Taky kóduje přirozená čísla. Využívá Fibonacciho posloupnost. Číslo, které chceme zakódovat rozložíme na součet čísel z Fibonacciho posloupnosti. Podle toho, která čísla jsme použili, tak je označíme **1** a když nepoužili tak **0**. Nikdy nesmíme použít 2 sousední čísla. Jedná se o prefixový kód.

Fibonnacciho kódy

Příklad 13

Chci zakódovat číslo 11

- Posloupnost: 1, 2, 3, 5, 8, 13, 21...
- $11 = 8 + 3$
- **0 0 1 0 1** (**0** pro 1, **0** pro 2, **1** pro 3, ...)

5.3.4 Golombovy kódy

Kódování se dá nastavit parametrem m . Číslo se rozdělí na 2 části – *quotient* a *remainder*.

$$q = \left\lfloor \frac{x}{m} \right\rfloor$$

$$r = x - qm$$

Část q se zapíše unárně jako jedničky následované nulou. Druhou část r zakódujeme klasicky binárně. Obě částí zřetěžíme jako **q_unarni + 0 + r_binarni**.

Golomobvy kódy

Příklad 14

Máme $m = 3$ a chceme zakódovat $x = 5$.

$$q = \left\lfloor \frac{5}{3} \right\rfloor = 1 \text{ a } r = 5 \pmod{3} = 2$$

Pro 1 máme v binární **1** a pro 2 **10**. Po použití oddělovací **0** dostaneme **1010**.

Používá se například v bezztrátové kompresi obrazu (JPEG-LS).

5.3.5 Riceovy kódy

Jedná se o speciální případ Golombových kódů, kde se vždy $m = 2^k$. Při implementaci toto přináší veliké zrychlení.

$$q = x \gg k$$

$$r = x \& (2^k - 1)$$

Máme $x = 13$ a $k = 2$, takže $m = 2^2 = 4$

- 13 je v binární soustavě **1101**
- $q = \mathbf{1101} \ll 2 = \mathbf{11}$ (3) a ta je unárně **1110** (0 se přidá na konec vždy)
- $r = \mathbf{1101} \ \& \ (2^2 - 1)_{[10]} = \mathbf{1101} \ \& \ (4 - 1)_{[10]} = \mathbf{1101} \ \& \ \mathbf{11} = \mathbf{0001}$ (kvůli $k = 2$ se vezmou pouze poslední 2 bity)
- výsledný zápis: **1110 + 01 = 111001**

6 Statistické metody

6.1 Tunstalovo kódování

Jedná se o kódování, která všem vstupním znakům přiřazuje kódové slovo stejné délky. Délka těchto kódových slov je dána parametrem k , pro který platí

$$k \leq \lceil \log_m n \rceil$$

kde n je velikost zdrojové abecedy a m je velikost kódové abecedy. Výhodou je, že chyba se nešíří v kódu dále a zůstane pouze v jediném slově.

Jde o statický nebo semi-adaptivní model, takže musíme mít pravděpodobnosti výskytu symbolů.

Enkodér

1. Na vstupu máme abecedu, pravděpodobnosti, k
2. Spočítáme m^k
3. Začínáme s množinou slov $T = A$
4. Vezmeme vždy nejpravděpodobnější symbol z T , odstraníme ho a uděláme jeho zřetězení se všemi ostatními symboly v A a výslednou množinu opět uložíme do T ¹
5. Celý proces rozšiřování T opakujeme dokud bude $|T| \leq m^k$

Dekodér

1. Přečte daný počet bitů k
2. V tabulce najde znak/slovo pro dané kódové slovo

6.2 Shannon-Fanovo kódování

Myšlenkou je kódovat podle pravděpodobností jednotlivých symbolů. Jedná se o binární prefixový kód, který se snažil být optimální.

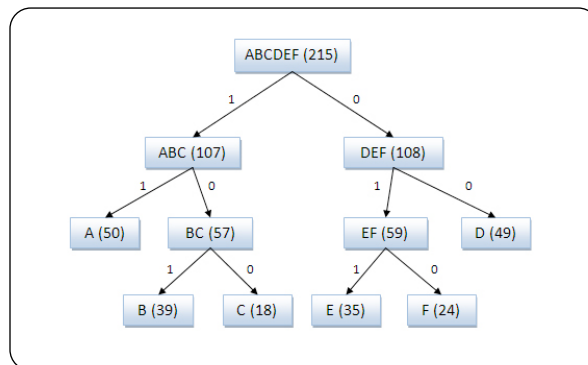
Postup kódování

1. seřazení všech symbolů podle jejich pravděpodobnosti výskytu sestupně
2. rozdělení seznamu na 2 části, aby části byly co nejvíc vyrovnané (pravděpodobnost každé se blížila 0.5)
3. první části se přiřadí bit **1** a druhé bit **0**
4. proces se opakuje pro každou vzniklou část dokud nedojdeme až na pravděpodobnosti jednotlivých symbolů

¹Vše ukládáme včetně pravděpodobností výskytu i pro dané n -gramy

a_i	$p(a_i)$	1	2	3	4	Code
a_1	0.36	0	00			00
a_2	0.18		01			01
a_3	0.18	1	10			10
a_4	0.12		11	110		110
a_5	0.09			111	1110	1110
a_6	0.07				1111	1111

Obrázek 1: Přiřazování bitů Shannon-Fanovo kódování tabulkou



Obrázek 2: Přiřazování bitů Shannon-Fanovo kódování stromem

Postup dekódování

1. na vstupu dostaneme datovou strukturu s přiřazením bitů a vstupní sekvenci
2. začíná se v kořeni stromu a podle 0 nebo 1 se jde doprava/doleva
3. ve chvíli kdy dojdeme do listu víme, že jsme přečetli daný symbol
4. vrátíme se do kořene a pokračujeme ve čtení vstupu, kde se skončilo

6.3 Huffmanovo kódování

Jedná se o prefixové kódování, které je pro daný model optimální.

Znaky jsou opět seřazeny dle pravděpodobnosti sestupně (pravděpodobnost symbolů tedy musíme znát ještě před začátkem kódování) a poslední 2 s nejmenší pravděpodobností se vždy „spojí“ do jediného a tomu se přiřadí 0 a 1 a neznámý zbytek řetězce. Takto se postupuje pořád iterativně než nám zbudou pouze 2 symboly a my můžeme ony neznáme řetězce rozvést. Symbol s nejčastějším výskytem má pak nejkratší kód.

Huffmanovo kódování průběh

Příklad 16

Začínáme se znaky $\{a_2, a_1, a_3, a_4, a_5\}$ a jejich pravděpodobnostmi 0.4, 0.2, 0.2, 0.1, 0.1. Pro každý znak a_i máme kódové slovo $c(a_i)$.

1. V prvním kroku spojíme a_4 a a_5 a přiřadíme jim kódová slova následovně²

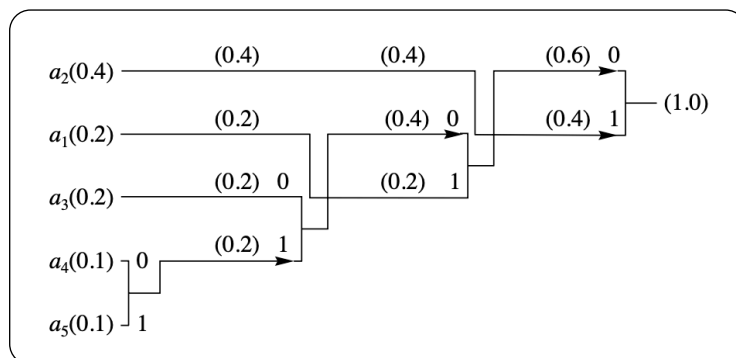
$$c(a_4) = \alpha_1 * 0$$

$$c(a_5) = \alpha_2 * 1$$

2. Tím vznikne nový znak a'_4 , který je spojením a_4 a a_5 . Pravděpodobnost tohoto znaku je součet pravděpodobností znaků, ze kterých vychází. Kódové slovo pro a'_4 je α_1 .
3. Dále se pokračuje od 1. kroku s novou abecedou A' , která je $\{a_2, a_1, a_3, a'_4\}$.

Sestavuje se strom podle toho, které znaky se kolikrát vyskytují. Strom obsahuje prázdný znak, jednotlivé znaky i různé kombinace. Uzel je tím blíže ke kořenu čím více má výskytů. Provádí se různé operace, které strom „upravují“.

²Znak * označuje spojení řetězců.



Obrázek 3: Postup sestavení Huffmanova stromu

Pokud bychom si měli strom na obrázku výše představit jako běžně nakreslený strom, tak pravá strana, kde je 1.0 bude kořen, ze kterého jdou dvě větve. Jedná přímo do listu a_2 a kódem **1** a další se větví.

znak	pravděpodobnost znaku	kódové slovo
a_1	0.2	01
a_2	0.4	1
a_3	0.2	000
a_4	0.1	0010
a_5	0.1	0011

Tabulka 1: Abeceda a její kódová slova podle Huffmanu.

Pokud tabulku s pravděpodobnostmi seřadíme různě (týká se případů kdy máme stejnou pravděpodobnost pro 2 symboly) můžeme dojít k různým kódům. Tyto kódy mají stejnou redundanci, ale délka kódových slov může být diametrálně odlišná.

Dekódování probíhá tak, že na vstupu dostaneme bitový proud a sestavený huffmanův strom. Na základě proudu procházíme strom a ve chvíli, kdy narazíme na list vypíšeme daný znak a pokračujeme dále opět od kořene.

Huffmanovo kódování se typicky kombinuje v návaznosti na jiné metody. Při implementaci je algoritmus velmi rychlý (má časovou složitost $O(n \cdot \log(n))$).

Theorem 17

Délka Huffmanova kódu je dána rovnicí

$$H(S) \leq \bar{l} < H(S) + 1$$

kde S je zdroj. Takže je omezena entropií zdroje a entropií zdroje navýšenou o **1** bit.

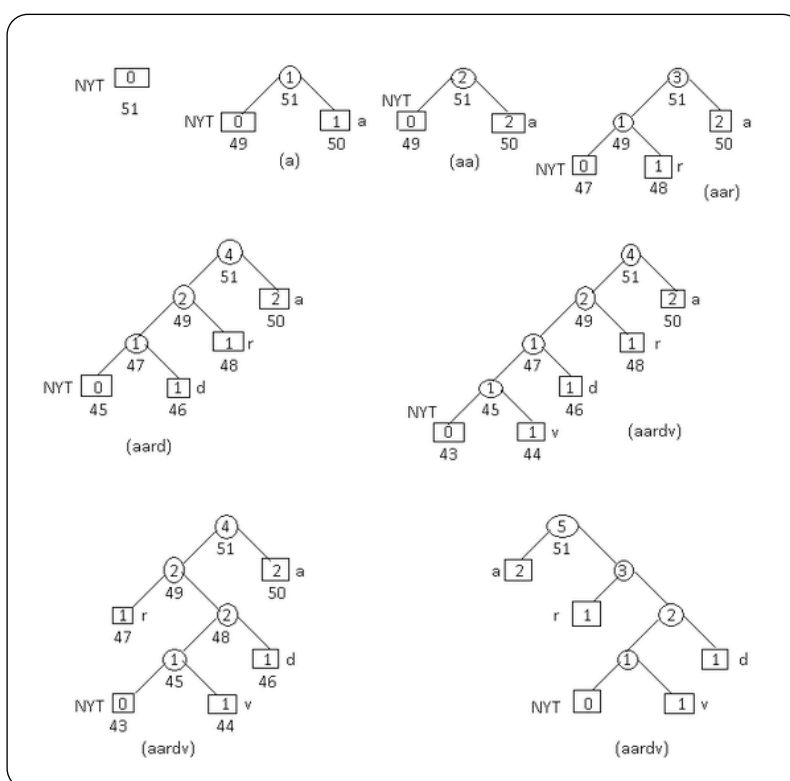
V tomto základním algoritmu počítáme s tím, že známe výskyt symbolů, případně se jedná o algoritmus, který vstup projde $2\times$ – jednou aby spočítal pravděpodobnosti symbolů a podruhé aby spočítal kódová slova. Pokud se tomuto chceme vyhnout musíme použít **adaptivní Huffmanovo kódování**, kde se strom neustále přepočítává a dynamicky upravuje na základě toho co jsme již přečetli.

6.3.1 Adaptivní Huffmanovo kódování

Musíme přidat *escape symbol* (reprezentuje ho NYT³ uzel), který je obsažen ve stromu hned na začátku a reprezentuje znak, který se ve stromu zatím nevyskytl. Při objevení nového symbolu je vyslán signál do NYT uzlu, který se „rozdělí“ na 2 potomky (nový symbol a NYT uzel). Vzhledem k tomu, že pravděpodobnosti se zjišťují za běhu je nutné strom v určitých případech přeskládat (zjištění *sibling property*).

Pokud již daný symbol ve stromu je, tak na vstup jde jeho kódové slovo, které se poskládá podle hran, kterými projde (obvykle **0** doleva a **1** doprava). Pokud symbol ve stromu není, tak jde na vstup kódové slovo NYT uzlu a daný symbol (může být libovolně předaný).

Oproti klasickému Huffmanovi jsou výsledná slova stejně optimální a s výjimkou jediného i stejná. Tou výjimkou je symbol, který je na stejné úrovni jako NYT list. Ten má v adaptivním modelu navíc poslední bit (obvykle jedničku). V abecedě je taky jeden symbol navíc a to NYT symbol (často taky ϵ).



Obrázek 4: Vývoj stromu v adaptivním Huffmanově kódování

Node number

Poznámka 18

Pokud máme abecedu velikosti n můžeme mít až $2n - 1$ uzlů. Začínáme číslovat od největšího (tzv. *node number*).

Kdy přeskládáme strom

Při každém přidání nového znaku z přepočítáme všechny hodnoty ve vnitřních uzlech (jsou součtem uzlů pod nimi). Uzel NYT má vždy hodnotu **0**, zbytek je určený podle toho kolikrát se který znak vyskytl. Pokud jsou na stejné úrovni stromu 2 hodnoty takové, že levý uzel má větší

³Not yet transmitted

hodnotu musí se prohodit s pravým uzlem (s menší hodnotou) na stejné úrovni. Tento postup se opakuje dokud nedojdeme až do kořene (klidně i vícekrát po přečtení jediného symbolu).

Obrázek se stromem výše nereflektuje přesně tuto skutečnost, protože ukazuje za mě o něco komplikovanější metodu.

6.4 Aritmetické kódování

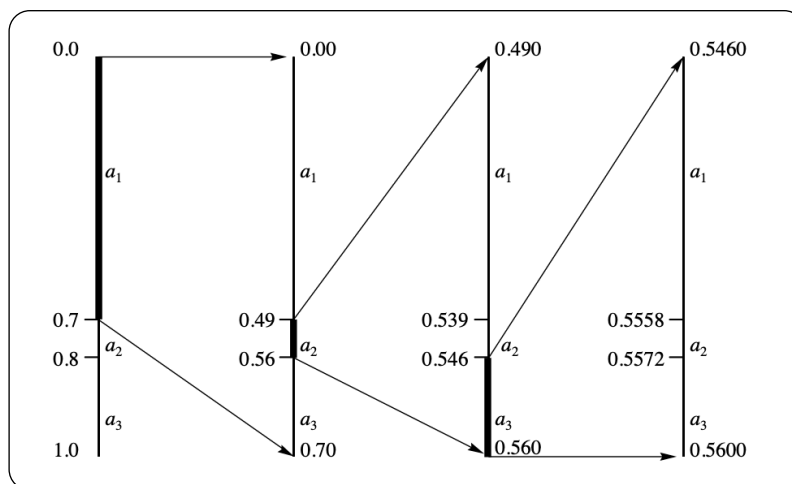
Jedná se o metodu, která je vhodná pro malé abecedy, kde je ovšem velký rozdíl v pravděpodobnostech symbolů. Narozdíl od Huffmana, kde každý symbol dostane kódové slovo, zde je celá zpráva zakódována do reálného čísla v intervalu $[0, 1)$.

Pro dělení intervalu se využívá **kumulativní distribuční funkce** definovaná jako

$$F_X(i) = \sum_{k=1}^i P(X = k)$$

Pro abecedu $A = \{a_1, \dots, a_n\}$ a její pravděpodobnosti $P = \{p_1, \dots, p_n\}$ pak máme

$$F_X(0) = 0; F_X(1) = p_1; F_X(2) = p_1 + p_2; \dots$$



Obrázek 5: Dělení intervalu při aritmetickém kódování

Jelikož v průběhu dělení intervalu bychom se dostávali stále do menších čísel, která mají delší desetinný rozvoj, tak z praktických důvodů se provádí přeškálování. Přeškálování se skládá z následujících 3 pravidel:

1. $L \leq U \leq \frac{1}{2}$ potom
 - $L = 2 \cdot L$ a $U = 2 \cdot U$
 - `print(01...1)`, kde c určuje počet **1**
 - nastav $c = 0$
2. $\frac{1}{2} \leq L \leq U$ potom
 - $L = 2(L - 0.5)$ a $U = 2(U - 0.5)$
 - `print(10...0)`, kde c určuje počet **0**
 - nastav $c = 0$
3. $\frac{1}{4} \leq L \leq U \leq \frac{3}{4}$ potom
 - $L = 2(L - 0.25)$ a $U = 2(U - 0.25)$
 - nastav $c = c + 1$

Tyto 3 pravidla můžeme opakovat v jediném případě dokud se nedostaneme na korektní hodnoty.

 *Níže zapsané postupy jsou vygenerovány Gemini.*

Algoritmus pro enkodér

- Na začátku jsou známé symboly a jejich pravděpodobnosti.
1. Příprava šuplíků: Rozdělíš startovní interval $[0,1)$ na části podle toho, jak moc jsou písmena častá (to je ta kumulativní funkce CDF). Čím častější písmeno, tím širší šuplík dostane.
 2. Čtení znaku: Přejde první písmeno. Podíváš se, kde jeho šuplík začíná (Low) a končí (High).
 3. Zúžení (Zoom): Zapomeneš na zbytek světa. Tvým novým světem se stává pouze tento šuplík.
 4. Opakování: V rámci tohoto nového malého výřezu znovu překreslíš šuplíky pro všechna písmena (ve stejném poměru) a přečteš další znak. znovu zoomuješ.
 5. Konec: Až text skončí, tvůj interval je neuvěřitelně malý (např. $[0.65312, 0.65315)$). Vybereš z něj jedno jediné číslo (třeba 0.65313) a to pošleš jako výsledný zkomprimovaný soubor.

Algoritmus pro dekodér

- Dekodér hraje opačnou hru. Dostane do ruky výsledné číslo (např. 0.65313) a zná stejná pravděpodobnosti jako enkodér.
1. Kam to padá? Dekodér se podívá na základní rozdělení intervalu $[0,1)$ a zeptá se: „Do čí hromádky padá číslo 0.65313?“
 2. Nalezení znaku: Zjistí, že číslo padá například do šuplíku písmene B. Dekodér okamžitě vypíše na výstup B.
 3. Zúžení (Zoom): Dekodér ví, že enkodér musel v tomto kroku skočit do šuplíku pro B. Udělá tedy stejný zoom jako předtím enkodér.
 4. Matematické odečtení (Normalizace): Aby dekodér mohl pokračovat, přepočítá pozici čísla 0.65313 v rámci tohoto nového, přiblíženého intervalu.
 5. Opakování: S novou polohou čísla se znovu zeptá: „A do čí hromádky padá teď?“ Najde další znak, vypíše ho a zoomuje dál.
 6. Konec: Dekodér takto pokračuje, dokud nenarazí na speciální znak konce souboru (EOF), nebo dokud nerozbalí přesný počet znaků, který mu enkodér předem nahlásil.

Poznámka 19

Při implementaci se využívá mapování intervalu na rozsah velkých celých čísel.

Jelikož pro počítač nás zajímá binární reprezentace, ta se udělá takto. Máme vstupní sekvenci x a pro ní číslo $T_x(x)$, které je z intervalu $[0,1)$ a je výstupem algoritmu. Vezmeme binární reprezentaci tohoto čísla a zkrátíme ji na délku $l(x)$, pro kterou platí $l(x) = \left\lceil \log \frac{1}{P(x)} \right\rceil + 1$ bitů, kde $P(x)$ je pravděpodobnost vstupní sekvence.

6.5 QM kódování

Jedná se o speciální variantu aritmetického kódování. Místo dělení na 2 symboly – **0** a **1**, se používá více pravděpodobný (MPS) a méně pravděpodobný symbol (LPS)⁴. Nepracuje se se vstupní abecedou jako s jednotlivými znaky, ale s jejich binární reprezentací.

⁴More probable symbol a less probable symbol.

Sledujeme pravděpodobnosti výskytu, které jsou q pro LPS a $1 - q$ pro MPS. Dále se používá dolní mez intervalu l a velikost intervalu A . Mohou nastat 2 situace:

1. Výskyt MPS – dolní mez se nemění ($l = l$) a interval se zmenší ($A = A - q$)
2. Výskyt LPS – dolní mez se posune ($l = l + A - q$) a velikost intervalu se nastaví přesně na velikost pro LPS ($A = q$)

Může dojít k operaci **přeskálování**, která se dělá při LPS a nastaví $l = 2(l - d)$ a $A = 2 \cdot A$.

Dnes se používá v nejnovějších standardech komprese videa (H.264 a H.265). Systém umí sám detekovat pokud by se LPS vyskytovalo častěji než MPS a symboly prohodit.

7 Kontextové metody

Platí, že výskyt symbolu na vstupu není nezávislý na vstupu ostatních symbolů. Pro modelování symbolu na vstupu se využije **kontext**, který má délku k (pro běžný text ideálně 5 nebo 6) a je pevně dána. **Kontext** je posloupnost bezprostředně předcházejících k symbolů⁵.

Maximální délka kontextu má vliv na sestavování datové struktury (nejčastěji *trie*). Musí tam být nějaká zastavující podmínka.

[Vložit obrázek jak vypadá *trie*]

7.1 PPM (Prediction with Partial Matching)

Využívá se zde **kontextu**. Pravděpodobnost výskytu symbolu se nezjišťuje pro všechny kontexty dané délky, ale pouze pro ty, které jsou na vstupu (tím se ušetří dost místa). Adaptivně modelujeme pravděpodobnost výskytu symbolů ve zkracujícím se kontextu jako posloupnosti bezprostředně předchozích symbolů.

Protože se někde musí začít, je zde *escape symbol* ε sloužící pro první výskyt nového symbolu.

V základním algoritmu se vždy snažíme dívat na nejdelší možný kontext a při neúspěchu ho postupně zkracujeme dokud nenarazíme na kontext, který můžeme použít nebo na to, že symbol se vyskytl poprvé (kontext -1) a pak ho zakódujeme s pravděpodobností $\frac{1}{M}$, kde M je velikost vstupní abecedy. Pro jediný krok tedy máme kontexty od k do -1 a v každém víme s jakou pravděpodobností jaký znak následoval.

Pro zvýšení efektivity se dá využít *principu exkluze*. Pokud kódér v kontextu délky k vypíše escape symbol, znamená to, že v aktuálním kontextu daný znak nebyl. Když dále přejde do kontextu $k - 1$ může z něj dočasně vyloučit všechny symboly z kontextu k . Pokud by totiž byl na vstupu některý z nich, dekoval by se již dříve.

Typy PPM metod

Máme několik typů, které se liší v tom jak modely odhadují pravděpodobnost escape symbolů. Tyto jednotlivé metody se označují PPMA, PPMB a PPMC.

1. u PPMA má escape symbol přiřazen pevnou četnost 1; neřeší zda-li z 1000 přečtených zanků bylo jen samé **a** nebo 50 různých znaků

$$P(\varepsilon) = \frac{1}{N+1} \quad P(i) = \frac{c_i}{N+1}$$

2. u PPMB je četnost escape symbolu určena podle četnosti již přečtených unikátních znaků (q)

⁵Jde tedy o Markovův model k -tého řádu.

$$P(\varepsilon) = \frac{q}{N} \quad P(i) = \frac{c_i - 1}{N}$$

3. u PPMC je velmi podobné verzi B, ale četnosti jednotlivých znaků se nesnižují (takže narůstá celkový součet)

$$P(\varepsilon) = \frac{q}{N + q} \quad P(i) = \frac{c_i}{N + q}$$

Čím je delší kontext, tím jsme schopni mít přesnou predikci, ale naopak máme vyšší míru vyslání escape symbolu. Nakládat s velkými kontexty je taky paměťově náročné (na to má vliv i zvolená datová struktura). Pro běžný text se volí $k = 5$ nebo $k = 6$.

7.2 PAQ

Jedná se o velmi moderní a pokročilý kompresní algoritmus. Uvažujme, že kontext nemusí být souvislá posloupnost předchozích symbolů. Můžeme použít libovolné předchozí symboly ze vstupu (předchozí vícebitová slova, nejvýznamnější bity předchozích slov, sousední slova více dimenzionálních dat). Kombinujeme různé modely (sledují různé vlastnosti) a také sledujeme různé kontexty, tomu říkáme **context mixing**.

Jako kontext se používají:

- bezprostředně předchozí bity (jako u PPM)
- nejvýznamnější bity předchozích slov
- bezprostředně předchozí vícebitová slova
- vybraná předchozí slova (*řídový kontext*)

Ke slučování více modelů se využívá neuronových sítí (od verze 7), které každému modelu přiřadí váhu. Předpovědi jsou pak podle vah s průměrovány a výstup je poslán do aritmetického kodéru.

S metodou PAQ se přišlo okolo roku 2000 (implementace). Má i různé verze (PAQAR či PAsQDa). Používají se i různé modely dat pro různé typy dat (jako text, obrázky, video).

7.3 Blokové třídění (Burrows-Wheelerova transformace)

Základem této metody je *Burrows-Wheelerova transformace*. Jedná se vlastně o permutaci, které přeskládá určité symbolu (stejně) více k sobě. V podstatě samo o sobě nekomprimuje, ale připravuje pro jiné metody, které pak dosáhnout velmi dobré komprese. Typicky pro RLE nebo MTF. Symboly se přeskládávají v jednotlivých **blocích**.

Velikost bloku je dána fixně (v praxi jde o stovky kB). Algoritmus probíhá vlastně ve 3 fázích – rotace jednotlivých bloků, lexikografické třídění bloků, vypsaní posledních symbolů setříděných bloků. Jediná rotace je takový posun, kde vezmeme první symbol a dáme ho nakonec. Počet rotací = počet znaků bloku. Dostaneme sekvenci o délce K a uděláme dalších $K - 1$ sekvencí. Třídění se provádí postupně podle pozic od 1. (§ delimitační znak má nejvyšší prioritu), při shodě na pozici se koukáme na další znak. Jako výstup vezeme sloupec posledních znaků po setřídění.

Permutace před setřazením		F	L
mississippi#		# mississipp	i
ississippi#m		i #mississip	p
ssissippi#mi		i ppi#missis	s
sissippi#mis		i ssippi#mis	s
issippi#miss		i ssissippi#	m
ssippi#missi	⇒	m ississippi	#
sippi#missis		p i#mississi	p
ippi#mississ		p pi#mississ	i
ppi#mississi		s ippi#missi	s
pi#mississip		s issippi#mi	s
i#mississipp		s sippi#miss	i
#mississippi		s sissippi#m	i

Obrázek 6: Setřizení transformací blokového třídění

Dekodér

1. Dekodér dostane pouze poslední sloupec L setřizené matice a číslo r , které říká na jakém řádku je posloupnost v originální
2. Když ho seřadí podle abecedy dostane první sloupec setřizené matice F
3. Díky vztahu mezi 1. a posledním znakem je dekodér schopný sestavit jednotlivé řetězce do původní podoby
4. Hledání znaků v řetězci probíhá způsobem:
 - začnu od znaku, který je na řádku r ve sloupci F
 - podle toho o jaké se jedná písmeno a kolikáté ono písmeno je v pořadí zde, jdu na stejný výskyt písmena ve sloupci L
 - vezmu toto pořadí ve sloupci (označené jako $L_{\text{pořadí}}$)
 - jdu na pozici $F_{\text{pořadí}}$ a ono písmeno zapíšu za předchozí
 - dále pokračuji od tohoto písmene

Blokové třídění

Příklad 20

- Dostaneme vstup **ANNB\$AA** a označíme jako sloupec L a číslo $r = 5$
- Seřadíme ho lexikograficky na **\$AAABNN** a dostaneme F
- Začínáme na 5. řádku, kde je první (a jediné) **B** a zapíšeme ho na výstup
- Toto **B** hledáme v L , kde má pozici 4
- Jdeme na 4. řádek v F , kde je třetí **A** a zapíšeme ho na výstup
- Hledáme třetí **A** v L , kde má pozici 7
- Jdeme na 7. řádek v F , kde je druhé **N** a zapíšeme ho na výstup
- Hledáme druhé **N** v L , kde má pozici 3
- Jdeme na 3. řádek v F , kde je druhé **A** a zapíšeme ho na výstup
- :

K samotnému zakódování výsledné sekvence se pak použije MTF, RLE či případné statické kódování jako je Huffmanovo.

Při reálné implementaci se data neposouvají a nestaví se matice, ale používají se pouze ukazatele. Vede to k výraznému šetření paměti. Mohou se využít i *suffix arrays*, což jsou pole, které seřadí všechny suffixy (konce) daného řetězce.

8 Slovníkové metody

Metody, které se spoléhají na to, že se symboly často vyskytují nebo nevyskytují a opakují se v určitých vzorech (**patterns**). Tyto vzory se v textu hledají a ukládají se do slovníku, na který se pak odkazuje. V komprimovaném textu se pak používá jen onen zakódovaný odkaz. Tento způsob (oproti statickým metodám) má dost jednoduchou a rychlou dekompresi.

Slovníky mohou být dané předem (např. n -gramy daného jazyka) a pak se jedná o statistickou metodu nebo se tvoří průběžně a jedná se o metodu adaptivní. Statistické metody můžou být známe např. pro zdrojový kód v různém jazyce či dokument v napsaném v $\text{\LaTeX}$. Tyto tabulky mají (nepřekvapivě) velmi odlišných obsah.

Kódování n -gramů

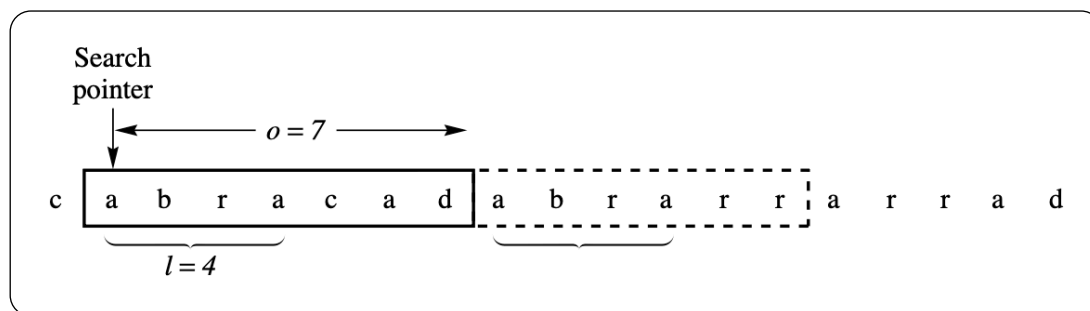
Ve statistickém přístupu se může používat i tzv. *kódování n -gramů*, kde máme dané n -gramy a k nim jejich kódy (sestavené třeba statistickým kódováním). Postupně procházíme text a bereme jednotlivé n -gramy a hledáme je v tabulce. Pokud ho tam najdeme tak zakódujeme a jdeme dál, pokud ne, tak zakódujeme pouze jediný symbol a o ten jediný se posuneme na další n -gram.

8.1 Rodina LZ77

Tato metoda má adaptivní slovník jako **posuvné okno**. První částí je K bezprostředně předchozích symbolů, které už kodér přečetl (něco jako kontext), nazývaných **search buffer**. Druhý buffer je **look-ahead** buffer, který se dívá na L následujících symbolů, které se budou kódovat. Celkem má posuvné okno velikost $K + L$. S prodlužujícím oknem je větší šance, že slovo najdeme, ale zvyšuje se náročnost.

Výstupem je vždy trojice (**offset**, **délka_shody**, **další_znak**).

- **offset** je vzdálenost mezi ukazatelem v search bufferu a look-ahead bufferem.
- **délka_shody** je počet stejných nalezených symbolů (vždy se snaží o co největší)
- **další_znak** je znak, který následuje po zakódovaném slově v search bufferu



Obrázek 7: Proces kódování v LZ77

Poznámka 21

Pro znaky které se nevyskytují nebo jsme na začátku a nemáme kontext bude trojice vypadat takto (**0**, **0**, **kódovaný_znak**).

Algoritmus muá různé varianty:

- **LZR (Rodeh)** – délky bufferů K a L jsou neomezené
- **LZSS** – vylepšuje neefektivitu kódování jediného znaku přidáním 1 bitového symbolu
- **Deflate** – bere výstupní data z LZSS a komprimuje je ještě pomocí Huffmana; používá se v ZIP nebo gzip

V praxi jsou délky bufferů tisíce symbolů (bytů). Pro posuvné okno se používá kruhová fronta, pro search buffer různé typy stromů. Protože LZW bylo patentováno narostla cena kvůli poplatkům. Využívá se v moderních protokolech jako HTTP či PNG a PDF.

8.2 Rodina LZ78

Řeší problémy LZ77, kde může nastat, že okno není dostatečně velké a nezvládne korektně pokrýt opakující se vzory (přitom někdy by ho stačilo zvětšit jen velmi zanedbatelně). Místo search bufferu tedy využívá pevně daný slovník. Tento slovník se musí vystavět jak u enkodéru, tak dekodéru a vždy se musí vybudovat stejně. Slovník se staví dynamicky v průběhu čtení vstupu.

Výstupem je dvojice (**i**, **c**), kde **i** je odkaz na index ve slovníku a **c** je další znak, který následuje po zakódované sekvenci. Pokud **i** = 0, pak se jedná o frázi, která zatím není ve slovníku. Každá tato nová fráze se přidá do slovníku a vypíšeme (0, **onen_znak**). Pokud už fráze ve slovníku je, zvýšíme *n*-gram o jeden a opět hledáme, dokud nenajdeme takový, který tam není a ten přidáme a vypíšeme (**index_ngramu**, **poslední_znak_fráze**) a pokračujeme od dalšího znaku.

Postup LZ78

Příklad 22

Máme vstup **BARBORAABARBARAUBARU** a postupně jedeme:

1. **B** - (0, **B**) - dict={1:**B**} - přečteno B
2. **A** - (0, **A**) - dict={1:**B**, 2:**A**} - přečteno BA
3. **R** - (0, **R**) - dict={1:**B**, 2:**A**, 3:**R**} - přečteno BAR
4. **B** - (1, 0) - dict={1:**B**, 2:**A**, 3:**R**, 4:**B0**} - přečteno BARBO
5. **R** - (3, **A**) - dict={1:**B**, 2:**A**, 3:**R**, 4:**B0**, 5:**RA**} - přečteno BARBORA
6. :

Výstupem by tedy byl slovník a zakódovány test jako [0**B**, 0**A**, 0**R**, 10, 3**A**, 2**B**, ...]

Tento postup teoreticky umožňuje mít až neomezeně velký slovník a zachytit různé vzory. V praxi ale neomezená velikost není možná a proto se při jeho naplnění celý vymaže a začne se budovat od znova. Datově se slovník reprezentuje jako *n*-ární strom. Jeho velikost bývá jednotky až desítky tisíc symbolů.

Varianta LZFG

JDe o kombinaci LZ77 a LZ78. Používá search buffer, ale implementuje ho efektivněji pomocí stromu. Tedy když přijde nový symbol nemusí se procházet celý seznam, ale projde se pouze strom a na výstup se dá identifikátor příslušného uzlu.

8.2.1 Varianta LZW

Jedná se o nejvíce populární variantu LZ78. Rozdíl je v tom, že na začátku je slovník inicializován tak, že obsahuje všechny symboly vstupní abecedy. Tudíž se nemusí myslet na to, jak zapisovat nové znaky, protože tam nikdy žádný nebude. Přidávají se tedy jen věci delší než 1. Tím je nám umožněno ve výstupu mít pouze index (čísla) bez znaků. Varianta LZC postupně zdvojnásobuje slovník a při naplnění do 2^{16} je statický. Varianta LZT udělá při naplnění to, že vyřadí slova, která mají nejdéle nekódovaný index.

Při implementaci se používá hashovací tabulky. Nativní příkaz **compress** na UNIXu využívá právě LZC metodu. LZW se používá ve formátu GIF.

Kódování je intuitivní. Již od prvního symbolu se přidávají další n -gramy (na začátku tedy dvojice).

Dekódování začíná se stejně inicializovaným slovníkem pouze o znacích abecedy a vstupními čísly. Do slovníku vždy přidáme zřetězení právě přečteného slova/symbolu a předchozího slova/symbolu. Stylem `slovo_predchozi + slovo_posledni`.

Přidávání do slovníku při dekódování

Příklad 24

- Původní vstup: `wabba_bwabb`
- Úvodní slovník: `[_ , a, b, o, w]` (je indexován od 1)
- Výstup po kódování: 5 2 3 3 2 1 6 8
- 1. Čteme 5; je ve slovníku (`w`); předtím nic $\Rightarrow$ nic nepřidáme
- 2. Čteme 2; je ve slovníku (`a`); předtím `w` $\Rightarrow$ přidáme `wa` (index 6)
- 3. Čteme 3; je ve slovníku (`b`); předtím `a` $\Rightarrow$ přidáme `ab` (index 7)
- 4. Čteme 3; je ve slovníku (`b`); předtím `b` $\Rightarrow$ přidáme `bb` (index 8)
- 5. :
- Po 4. kroku vypadá slovník takto `[_ , a, b, o, w, wa, ab, bb]`